Todo Management Application

Revature — Full-Stack SDLC Project

System Documentation

The DJs

Prepared by
Jaehoon Song
Daniel Patience
Jacqueline Lainhart

Document Control

Revision History

Per IEEE 830 and ISO document-control practice, all revisions are recorded below.

Version	Date	Author	Description
0.1	10 June 2026	The DJs	LaTeX structure, task breakdown, ERD, Definition of Done, and Task Backlog
0.2	11 June 2026	The DJs	API Contract HTTP request/response trace documentation (payloads and status codes)
0.3	12 June 2026	The DJs	Phase 2 complete: wireframes, component design, architecture and security planning, LaTeX module refactor
0.4	16 June 2026	The DJs	Phase 3 start: Spring Boot project scaffolding, Kiro steerings, ignore rules
0.5	19 June 2026	The DJs	Phase 3 backend: JPA entities, password validation, Kiro specs for registration and auth
0.6	22 June 2026	The DJs	Phase 3 complete: registration, task/subtask CRUD, specs verified
0.7	23 June 2026	The DJs	Phase 4 start: Angular auth integration, dashboard specs, frontend scaffolding
0.8	24 June 2026	The DJs	Phase 4 implementation: full frontend with auth, dashboard, services
0.9	25 June 2026	The DJs	Phase 4 complete: UI/UX polish
1.0	26 June 2026	The DJs	Phase 5: API contract synced to implementation, database ERD updated, all tests passing, presentation ready

Document Information

Field	Value
Document type	System Documentation (5-phase SDLC)
Organization	Revature
Project	Todo Management Application — Full-Stack SDLC
Team	The DJs
Authors	Jaehoon Song, Daniel Patience, Jacqueline Lainhart
Intended audience	Development team and Revature instructors (Trainers)
Module structure	Requirements / Behavioral / Structural / Team Work Records; `appendix/A01-A05`

Contents

1 Project Overview

1.1 Objective

Build a full-stack, secure Todo Management application as part of the Revature full-stack SDLC training program. The team moves from initial requirement analysis to a functional product demonstration, simulating a real-world Software Development Lifecycle (SDLC).

1.2 Technology Stack

Layer	Technology
Frontend	Angular
Backend	Java (Spring Boot, Web, Data JPA)
Database	SQLite
Build / Tools	Gradle, Git, GitHub

1.3 User Stories (Scope)

The following stories define *what* the system must deliver. Each phase builds on the prior phase toward a complete implementation.

1. **Registration:** As a new user, I can register an account.
2. **Authentication:** As a user, I can log in and log out securely.
3. **Task Management:** As a user, I can create, read, update, and delete primary Todo items.
4. **Organization:** As a user, I can create, read, update, and delete nested subtasks.

1.4 Project Roadmap

Phase	Name	Goal
1	Requirements Analysis	Deconstruct user stories; define schema and API contracts.
2	Design	Plan architecture, wireframes, and security flows.
3	Backend Development	Build REST API, security layer, and persistence.
4	Frontend Development	Build Angular UI and integrate with the API.
5	Presentation	Demonstrate working software and architecture.

Requirements Analysis Document

Todo Management Application

Revature — Full-Stack SDLC Project

The DJs

Prepared by

Jaehoon Song
Daniel Patience
Jacqueline Lainhart

Document ID: TMA-RAD-P1
Version: 0.3
Status: Draft
Issue date: June 26, 2026

2 Requirements Analysis

This portion decomposes the project user stories into granular technical tasks and defines the REST API contract—the agreed communication standard between the Angular frontend and the Spring Boot backend. A use case diagram summarizes each story as an actor–system interaction; the task breakdown and endpoint specifications establish what the system must expose before implementation begins.

2.1 Task Breakdown

2.1.1 Epic 1: Account Creation

User story: As a new user, I can register an account to start tracking my todo tasks.

Task ID	Description
AUTH-01	Define `User` entity fields (id, username, email, password); SQLite `users` table.
AUTH-02	Encrypt passwords before storage (persist hash only, never plaintext).
AUTH-03	Define validation rules (unique username/email, required fields).
AUTH-04	Specify registration endpoint: `POST /api/auth/register`.
AUTH-05	Document responses: HTTP 200 on success; HTTP 409 duplicate username/email; HTTP 400 on validation errors.

2.1.2 Epic 2: Authentication

User story: As a user, I can log in and log out to securely access my todo items.

Task ID	Description
AUTH-06	Verify username/password combination against the database on login.
AUTH-07	Specify login endpoint: `POST /api/auth/login`.
AUTH-08	Return HTTP 200 when request JSON matches stored credentials.
AUTH-09	Define session-based authentication for protected routes.
AUTH-10	Document logout behavior and HTTP 401 responses for unauthenticated access.

2.1.3 Epic 3: Task Management

User story: As a user, I can create, edit, and delete todo items to keep track of my work.

Task ID	Description
TODO-01	Define `Todo` entity (id, title, description, completed, user_id, timestamps).
TODO-02	Specify `POST /api/todos` — create todo.
TODO-03	Specify `GET /api/todos` — list todos for authenticated user.
TODO-04	Specify `GET /api/todos/{id}` — retrieve single todo.
TODO-05	Specify `PUT /api/todos/{id}` — update todo fields.
TODO-06	Specify `DELETE /api/todos/{id}` — delete todo (cascade subtasks).
TODO-07	Enforce ownership: users may only access their own todos.

2.1.4 Epic 4: Subtask Organization

User story: As a user, I can create, edit, and delete subtask items to better organize my primary tasks.

Task ID	Description
SUB-01	Define `Subtask` entity (id, title, completed, todo_id, timestamps).
SUB-02	Use `GET /api/todos/{id}` to retrieve the parent todo before subtask operations.
SUB-03	Specify `POST /api/todos/{id}/{subtask}` — create subtask.
SUB-04	Specify `PATCH /api/todos/{id}/{subtask}` — update subtask fields.
SUB-05	Specify `DELETE /api/todos/{id}/{subtask}` — delete subtask.
SUB-06	Validate parent todo exists, belongs to the authenticated user; document cascade delete when parent todo is removed.

2.2 API Contract

All endpoints follow RESTful conventions with JSON request/response bodies and standard HTTP status codes. Each exchange shows a raw HTTP request or response trace: status line, headers, a blank line, then the body. All paths are relative to the API base URL (`http://localhost:8080`).

Authentication. Protected endpoints require an `Authorization: Bearer <token>` header. The token is a signed JWT issued by `POST /api/auth/login`. Requests without a valid token receive `HTTP 401`.

Ownership. Users can only access their own resources. Attempting to read, update, or delete another user's task or subtask returns `HTTP 403 Forbidden`.

DELETE responses. A successful delete returns `HTTP/1.1 204 No Content` with no response body—the server has fulfilled the request and there is no representation to return (RFC 9110). Error cases (e.g. resource not found) return `4xx` with a JSON body as documented below.

2.2.1 Endpoint Summary

Method	Path	Purpose	Auth
POST	/api/auth/register	Register a new account	Public
POST	/api/auth/login	Authenticate, receive JWT	Public
POST	/api/todos	Create a todo	Bearer
GET	/api/todos	List todos for authenticated user	Bearer
GET	/api/todos/{id}	Retrieve a single todo	Bearer
PUT	/api/todos/{id}	Update a todo	Bearer
DELETE	/api/todos/{id}	Delete a todo (cascade subtasks)	Bearer
GET	/api/todos/{id}/subtasks	List subtasks for a todo	Bearer
POST	/api/todos/{id}/subtasks	Create a subtask	Bearer
GET	/api/todos/{id}/subtasks/{subtaskId}	Retrieve a single subtask	Bearer
PUT	/api/todos/{id}/subtasks/{subtaskId}	Update a subtask	Bearer
DELETE	/api/todos/{id}/subtasks/{subtaskId}	Delete a subtask	Bearer

2.2.2 Account Creation

Use the resource-creation flow template for this `POST` endpoint (Figure 1).

`POST /api/auth/register`

```
POST /api/auth/register HTTP/1.1
Content-Type: application/json

{
    "username": "String (5-18 chars)",
    "password": "String (8+ chars,
     uppercase, lowercase, digit, special)"
}
```

REQ ⟶ ⟵ RESP

```
HTTP/1.1 201 Created
(empty body)

HTTP/1.1 400 Bad Request
Content-Type: text/plain

Username must not be blank.
Username must be between 5 and 18
     characters.
Password must not be blank.
Password must contain at least one special
     character (!@#$%^&*).
Username 'example' is already taken.

HTTP/1.1 409 Conflict
Content-Type: text/plain

Could not complete registration: data
     conflict
```

2.2.3 Authentication

Use the login/auth flow template for this `/auth` endpoint (Figure 2).

`POST /api/auth/login`

```
POST /api/auth/login HTTP/1.1
Content-Type: application/json

{
    "username": "String",
    "password": "String"
}
```

REQ ⇄ RESP

```
HTTP/1.1 200 OK
Authorization: Bearer <signed-jwt-token>
Content-Length: 0
(empty body)

HTTP/1.1 401 Unauthorized
Content-Type: text/plain

Invalid username or password
```

JWT Claims: The issued token contains `sub` (username), `userId` (UUID string), `iat` (issued-at epoch seconds), and `exp` (expiration = iat + 24 hours).

2.2.4 Task Management

Use the resource read, creation, and delete flow templates (Figures 3, 4, and 5).

All task endpoints require `Authorization: Bearer <token>`. The authenticated user's UUID is extracted from the JWT and used for ownership enforcement.

`POST /api/todos`

```
POST /api/todos HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
    "title": "String",
    "completed": boolean
}
```

REQ ⇄ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "id": "UUID",
    "userId": "UUID",
    "title": "String",
    "completed": boolean
}

HTTP/1.1 400 Bad Request
Content-Type: application/json

{
    "status": 400,
    "message": "Task title must not be
     blank."
}
```

GET /api/todos

```
GET /api/todos HTTP/1.1
Authorization: Bearer <token>
```

REQ ⟶
⟵ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

[
    {
        "id": "UUID",
        "userId": "UUID",
        "title": "String",
        "completed": boolean
    }
]
```

GET /api/todos/{id}

```
GET /api/todos/{id} HTTP/1.1
Authorization: Bearer <token>
```

REQ ⟶
⟵ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "id": "UUID",
    "userId": "UUID",
    "title": "String",
    "completed": boolean
}

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404,
    "message": "Task not found: <id>"
}

HTTP/1.1 403 Forbidden
Content-Type: application/json

{
    "status": 403,
    "message": "User <userId> does not own
     task <taskId>"
}
```

PUT /api/todos/{id}

```
PUT /api/todos/{id} HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
    "title": "String",
    "completed": boolean
}
```

REQ ⟶
⟵ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "id": "UUID",
    "userId": "UUID",
    "title": "String",
    "completed": boolean
}

HTTP/1.1 400 Bad Request
Content-Type: application/json

{
    "status": 400,
    "message": "Task title must not be
     blank."
}

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404,
    "message": "Task not found: <id>"
}

HTTP/1.1 403 Forbidden
Content-Type: application/json

{
    "status": 403,
    "message": "User <userId> does not own
     task <taskId>"
}
```

DELETE /api/todos/{id}

Deletes the todo and cascade-deletes all its subtasks first.

```
DELETE /api/todos/{id} HTTP/1.1
Authorization: Bearer <token>
```

REQ ⟶
⟵ RESP

```
HTTP/1.1 204 No Content

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404,
    "message": "Task not found: <id>"
}

HTTP/1.1 403 Forbidden
Content-Type: application/json

{
    "status": 403,
    "message": "User <userId> does not own
     task <taskId>"
}
```

2.2.5 Subtask Organization

All subtask endpoints are nested under `/api/todos/{id}/subtasks` and require `Authorization: Bearer <token>`. Every operation first verifies that the parent task exists and belongs to the authenticated user.

Use the resource read, creation, and delete flow templates (Figures 6, 7, and 8).

`GET /api/todos/{id}/subtasks`

```
GET /api/todos/{id}/subtasks HTTP/1.1
Authorization: Bearer <token>
```

REQ ⟶ / ⟵ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

[
    {
        "id": "UUID",
        "taskId": "UUID",
        "title": "String",
        "completed": boolean
    }
]

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404,
    "message": "Task not found: <id>"
}

HTTP/1.1 403 Forbidden
Content-Type: application/json

{
    "status": 403,
    "message": "User <userId> does not own
     task <taskId>"
}
```

POST /api/todos/{id}/subtasks

```
POST /api/todos/{id}/subtasks HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
    "title": "String",
    "completed": boolean
}
```

REQ ⇄ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "id": "UUID",
    "taskId": "UUID",
    "title": "String",
    "completed": boolean
}

HTTP/1.1 400 Bad Request
Content-Type: application/json

{
    "status": 400,
    "message": "Subtask title must not be
     blank."
}

{
    "status": 400,
    "message": "Cannot add subtasks to a
     completed task."
}

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404,
    "message": "Task not found: <id>"
}
```

GET /api/todos/{id}/subtasks/{subtaskId}

```
GET /api/todos/{id}/subtasks/{subtaskId}
     HTTP/1.1
Authorization: Bearer <token>
```

REQ ⇄ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "id": "UUID",
    "taskId": "UUID",
    "title": "String",
    "completed": boolean
}

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404,
    "message": "Subtask not found:
     <subtaskId>"
}
```

`PUT /api/todos/{id}/subtasks/{subtaskId}`

```
PUT /api/todos/{id}/subtasks/{subtaskId}
    HTTP/1.1
Content-Type: application/json
Authorization: Bearer <token>

{
    "title": "String",
    "completed": boolean
}
```

REQ ⇄ RESP

```
HTTP/1.1 200 OK
Content-Type: application/json

{
    "id": "UUID",
    "taskId": "UUID",
    "title": "String",
    "completed": boolean
}

HTTP/1.1 400 Bad Request
Content-Type: application/json

{
    "status": 400,
    "message": "Subtask title must not be
     blank."
}

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404,
    "message": "Subtask not found:
     <subtaskId>"
}
```

`DELETE /api/todos/{id}/subtasks/{subtaskId}`

Removes one subtask from the parent todo. On success the server responds with `204 No Content` and an empty body; no row count or deleted entity is returned.

```
DELETE
    /api/todos/{id}/subtasks/{subtaskId}
    HTTP/1.1
Authorization: Bearer <token>
```

REQ ⇄ RESP

```
HTTP/1.1 204 No Content

HTTP/1.1 404 Not Found
Content-Type: application/json

{
    "status": 404,
    "message": "Subtask not found:
     <subtaskId>"
}
```

2.2.6 Common Error Responses

Status	Content-Type	Meaning
400	`application/json` or `text/plain`	Validation failure (blank title, weak password, etc.)
401	`text/plain`	Missing, malformed, or expired token
403	`application/json`	Authenticated user does not own the requested resource
404	`application/json`	Resource (task or subtask) does not exist
409	`text/plain`	Data conflict (duplicate username at DB level)
503	`text/plain`	Database temporarily unavailable

JSON error body format (for 400, 403, 404 from protected endpoints):

```
{
    "status": <http-status-code>,
    "message": "<human-readable description>"
}
```

Plain text error body (for 401, registration 400, 409, 503): the response body is a single plain-text string describing the error.

Behavioral Specification Document

Todo Management Application

Revature — Full-Stack SDLC Project

The DJs

Prepared by

Jaehoon Song
Daniel Patience
Jacqueline Lainhart

Document ID: TMA-BEH-P1
Version: 0.3
Status: Draft
Issue date: June 26, 2026

3 Behavioral Specification

This portion documents how actors and application layers interact during key operations. A state diagram captures the client login/session lifecycle; sequence diagrams model each user story as a flow of requests and responses, including success paths and error handling; UI wireframes map the same stories to screens, routes, and navigation so the frontend team can implement against a shared behavioral model.

3.1 Sequence Diagrams

The team modeled each user story as a request–response flow across five layers: Client/Requester, API/Interface Layer, Business Logic Layer, Data Access Layer, and Data Storage. Each diagram aligns with the REST endpoints in Section 2.2 and documents both success paths and error handling. Diagrams are grouped by epic below.

3.1.1 Epic 1: Account Creation

User story: As a new user, I can register an account to start tracking my todo tasks.
`POST /api/auth/register` — duplicate username/email returns `409`; validation errors return `400`. Figure 1 documents the resource creation flow: Scenario A persists a new account when the username is unique; Scenario B returns a conflict when the resource already exists.

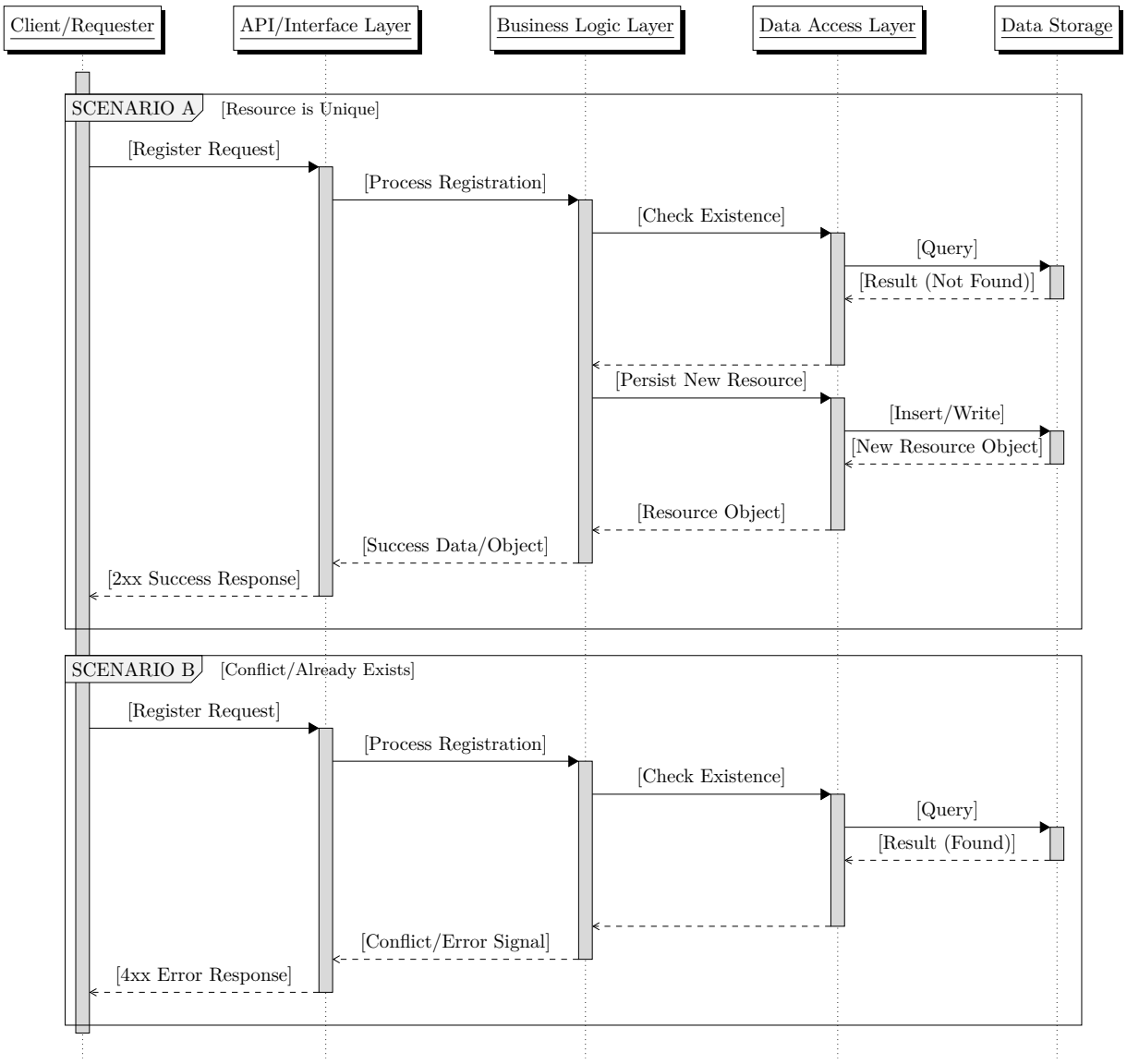


Figure 1: Account creation — `POST /api/auth/register`

3.1.2 Epic 2: Authentication

User story: As a user, I can log in and log out to securely access my todo items.
`POST /api/auth/login` — invalid credentials return `401`. Figure 2 documents the login/auth flow: Scenario A validates credentials and returns a token; Scenario B rejects invalid credentials.

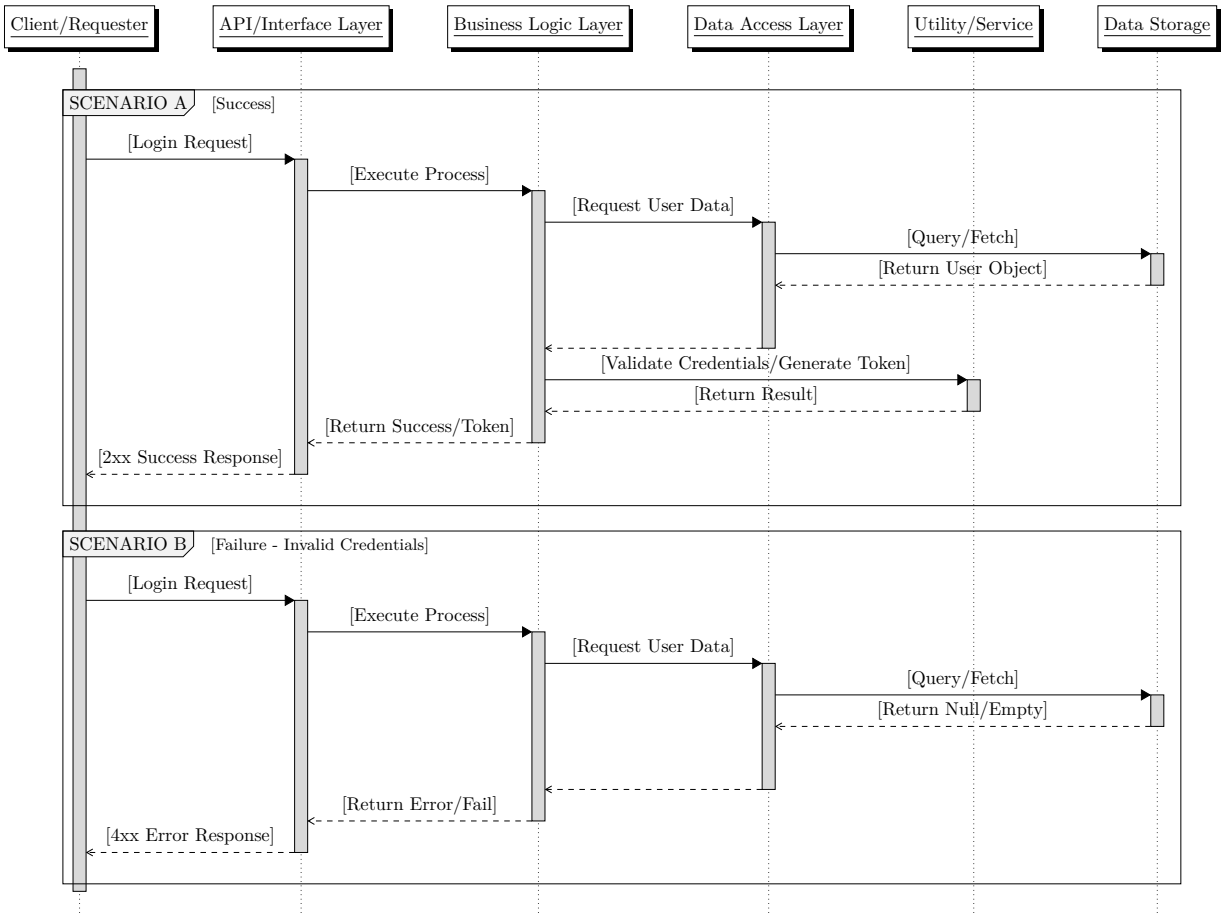


Figure 2: Authentication — `POST /api/auth/login`

3.1.3 Epic 3: Task Management

User story: As a user, I can create, edit, and delete todo items to keep track of my work.
`GET /api/todos` lists todos for the authenticated user; `POST /api/todos` and `PUT /api/todos/{id}` create or edit a todo; `DELETE /api/todos/{id}` removes a todo and cascades its subtasks. Validation errors return `400`; missing resources return `404`. Figure 3 documents the read flow; Figure 4 documents create and edit with success and null-result error paths; Figure 5 documents delete.

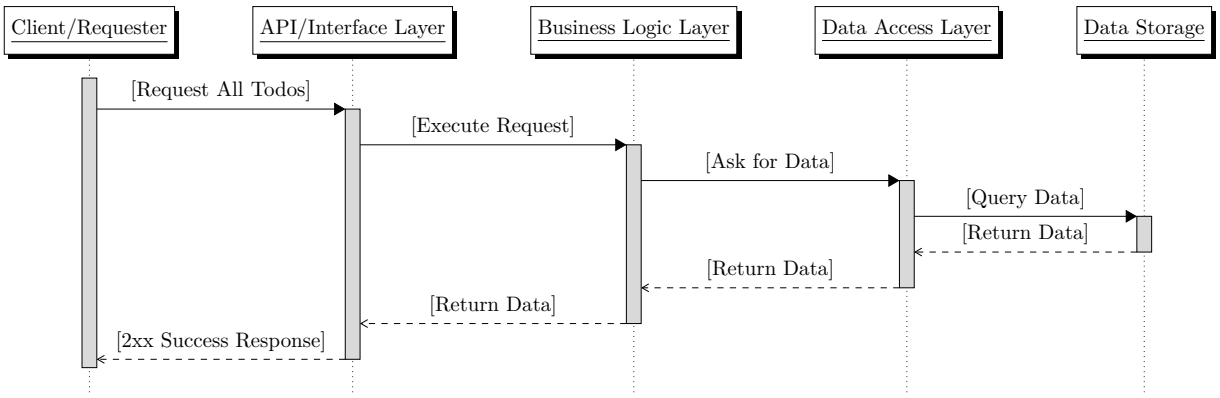


Figure 3: List todos — `GET /api/todos`

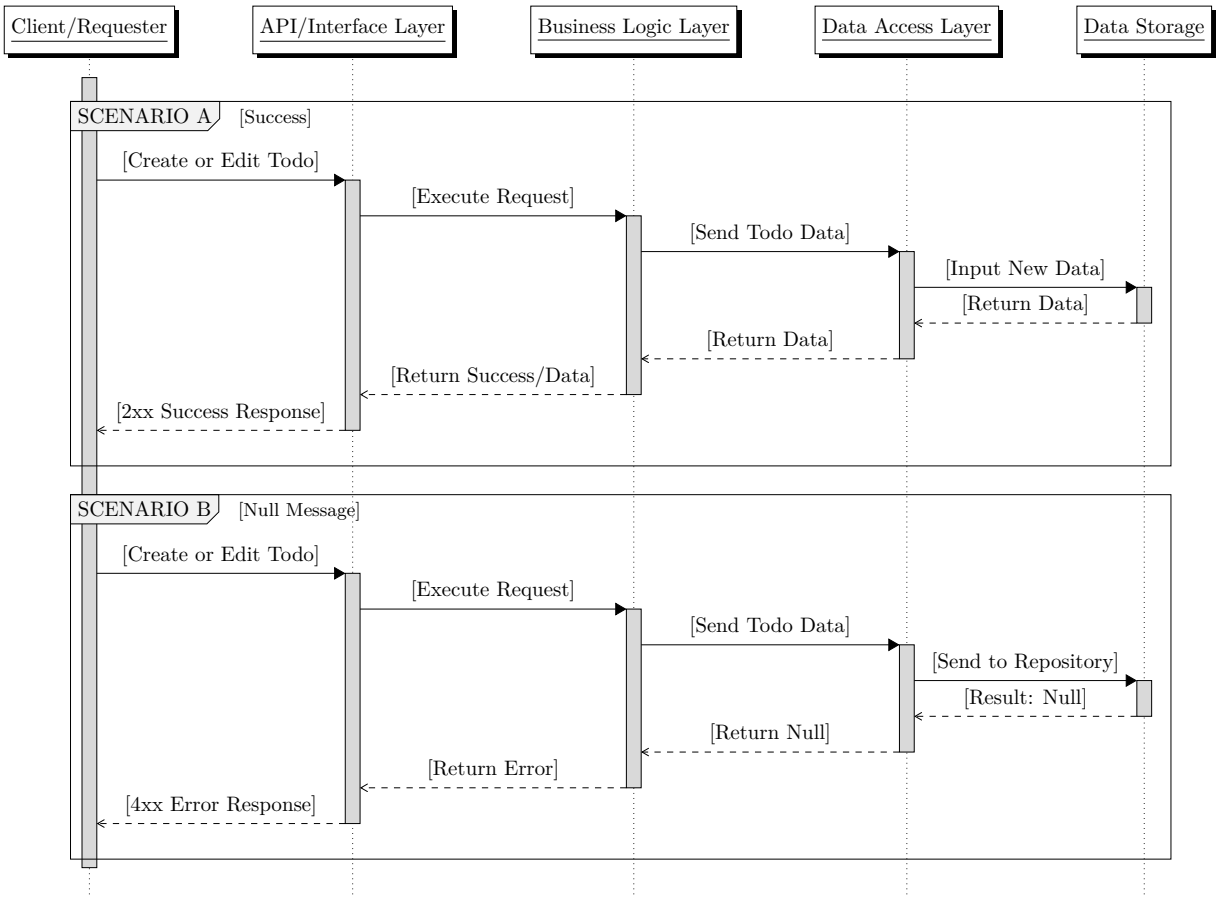


Figure 4: Create or edit todo — `POST /api/todos`, `PUT /api/todos/{id}`

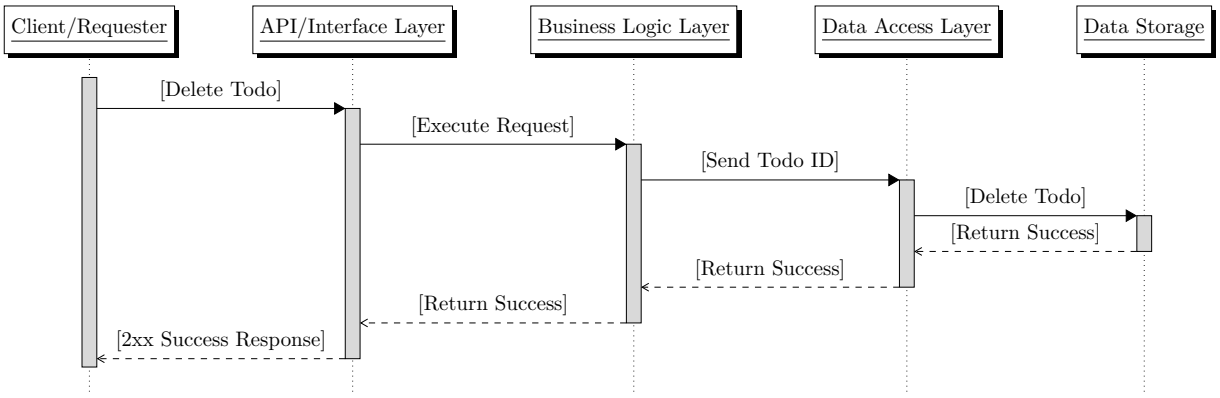


Figure 5: Delete todo — `DELETE /api/todos/{id}` (API contract: `204 No Content`)

3.1.4 Epic 4: Subtask Organization

User story: As a user, I can create, edit, and delete subtask items to better organize my primary tasks. `GET /api/todos/{id}` retrieves the parent todo before subtask operations; `POST /api/todos/{id}/{subtask}` and `PUT /api/todos/{id}/{subtask}` create or edit a subtask; `DELETE /api/todos/{id}/{subtask}` removes one subtask. The parent todo must exist and belong to the authenticated user. Figure 6 documents the read flow in parent-todo context; Figure 7 documents create and edit with success and null-result error paths; Figure 8 documents delete.

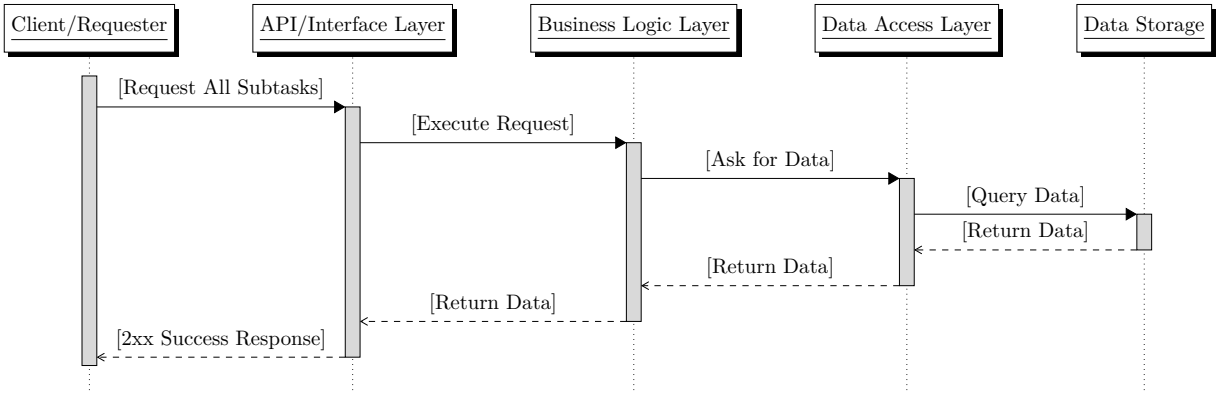


Figure 6: List subtasks — `GET /api/todos/{id}` (parent todo context)

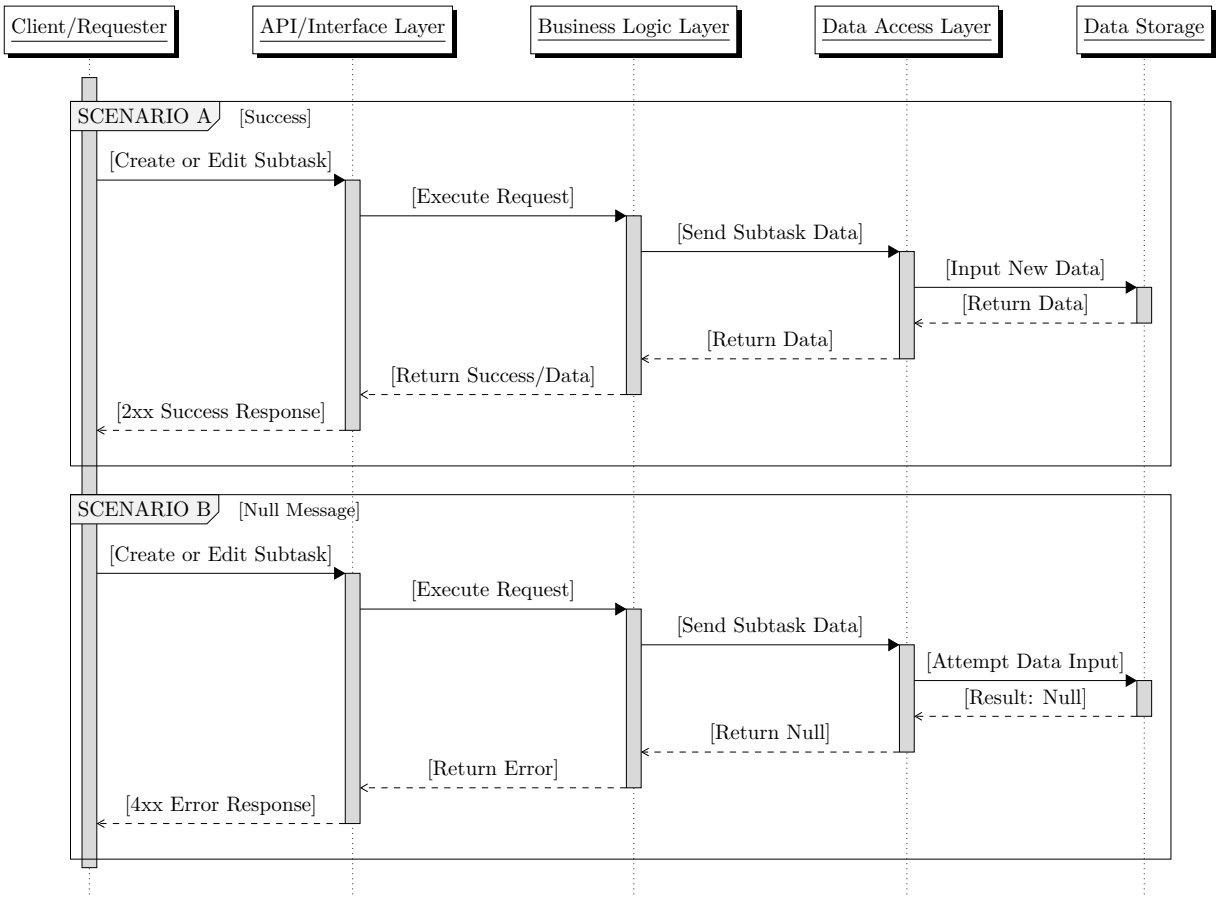


Figure 7: Create or edit subtask — `POST /api/todos/{id}/{subtask}`, `PUT /api/todos/{id}/{subtask}`

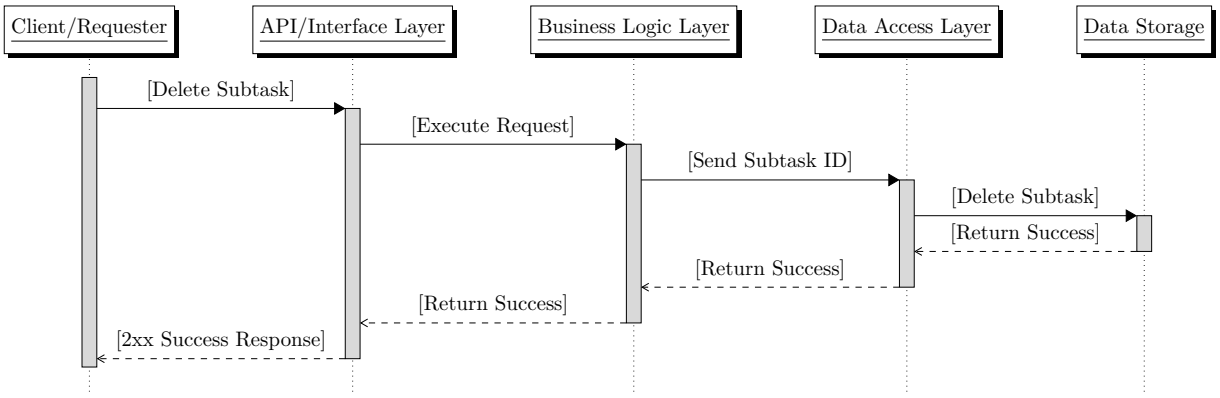


Figure 8: Delete subtask — `DELETE /api/todos/{id}/{subtask}` (API contract: `204 No Content`)

3.2 UI Wireframing

Low-fidelity wireframes define the primary screens and navigation paths for the Angular frontend. Each view maps to the user stories in Section 1; arrows in the interaction overview show how the client moves between routes after register, login, logout, and dashboard actions.

3.2.1 Screen Navigation

Figure 9 summarizes the application flow. Unauthenticated users land on the home (login) screen; **SIGN UP** routes to register and **LOG IN** or successful **CREATE ACCOUNT** routes to the dashboard. **LOG OUT** returns the user to login.

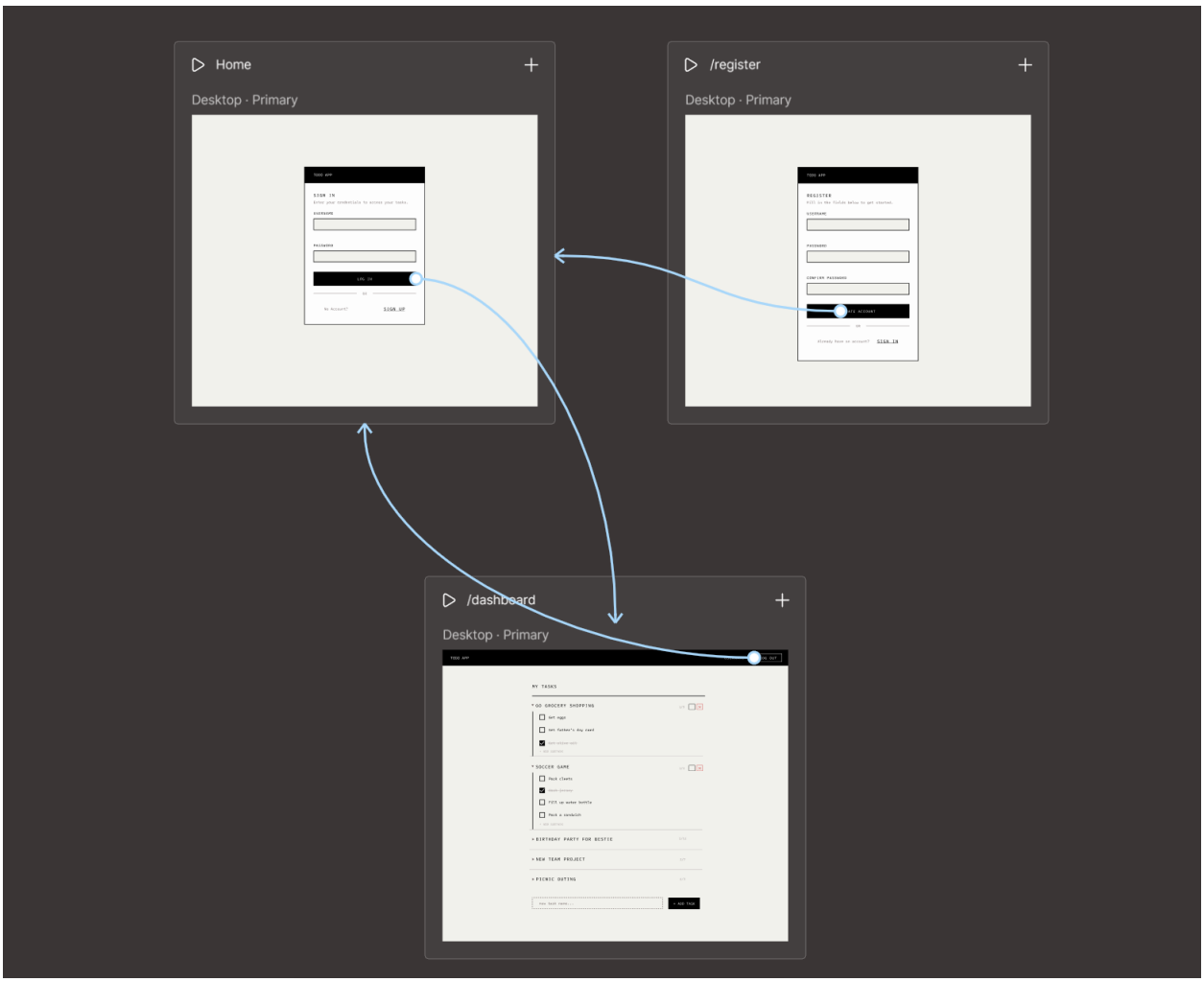
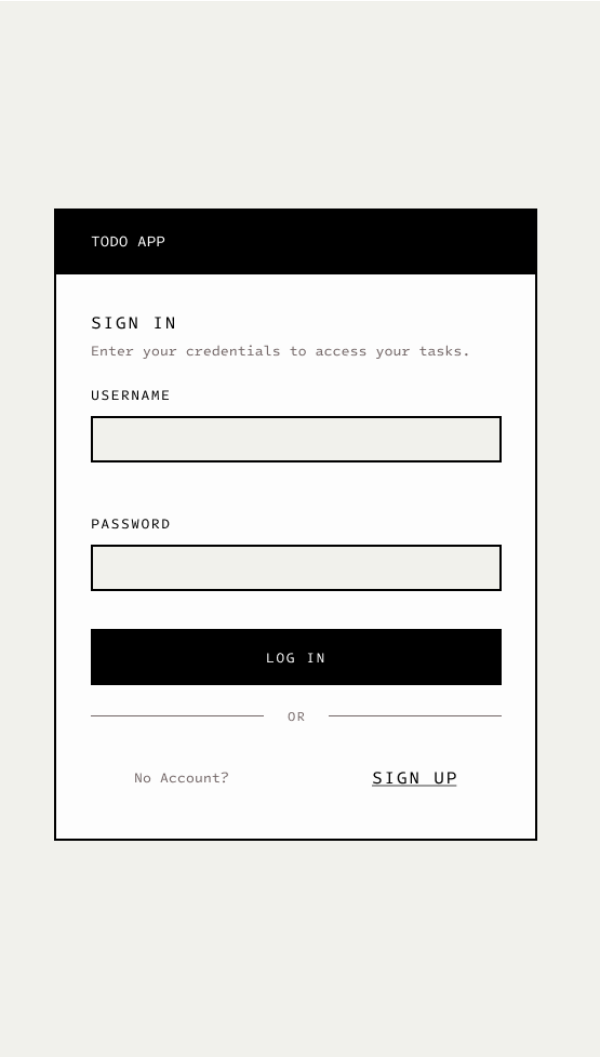


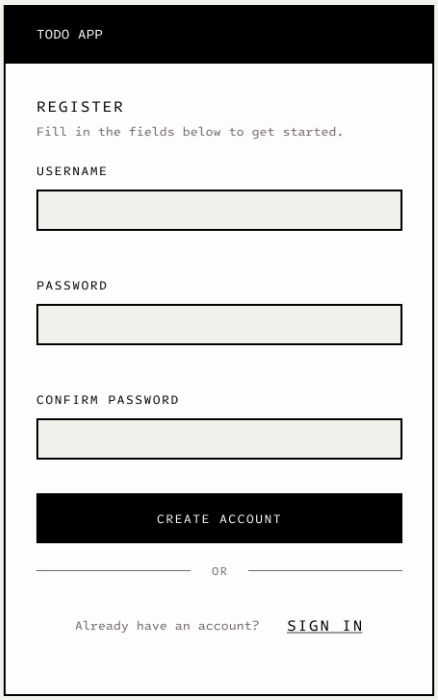
Figure 9: Screen navigation — home, register, and dashboard

3.2.2 Authentication Screens

Figures 10a and 10b cover Epic 1 (account creation) and Epic 2 (authentication). Both screens share the `TODO APP` header and a footer link to switch between sign-in and sign-up.



(a) Login — home route (`/`)



(b) Register — `/register`

3.2.3 Task Dashboard

Figure 11 covers Epic 3 (task management) and Epic 4 (subtask organization). The dashboard lists primary todos with expandable groups, completion progress (e.g. `1/3`), checkboxes, and delete controls. Users add top-level tasks via **+ ADD TASK** and nested items via **+ ADD SUBTASK**. The header shows the authenticated username and a **LOG OUT** action.

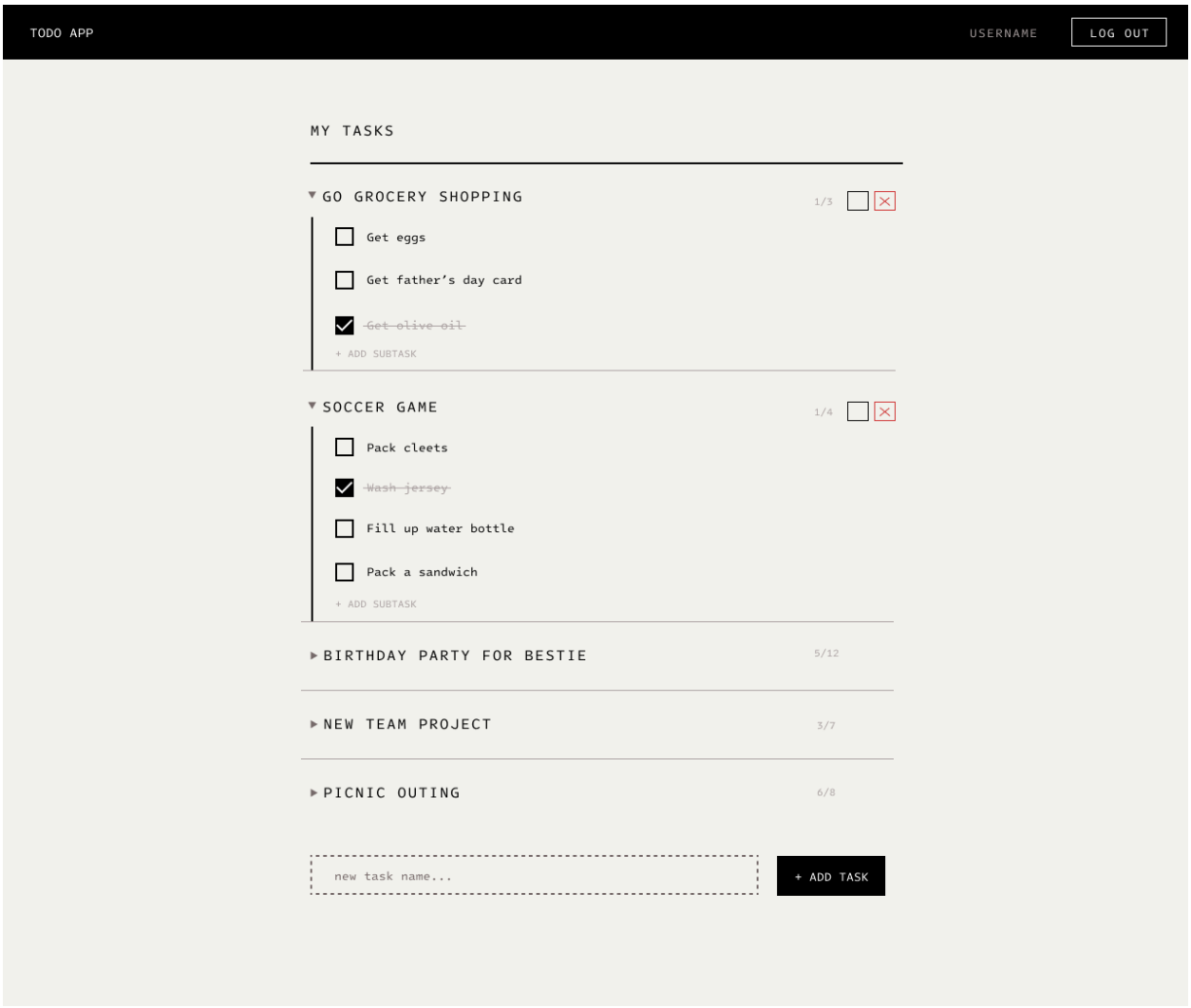


Figure 11: Dashboard — `/dashboard`

Structural Specification Document

Todo Management Application

Revature — Full-Stack SDLC Project

The DJs

Prepared by

Jaehoon Song
Daniel Patience
Jacqueline Lainhart

Document ID: TMA-STR-P1
Version: 0.3
Status: Draft
Issue date: June 26, 2026

4 Structural Specification

This portion defines the persistent data model and system structure for the Todo Management application. The component diagram, entity summary, Crow's Foot ERD, JPA class diagram, data dictionary, and integrity rules describe how the application is deployed and how `User`, `Todo`, and `Subtask` data is organized and constrained in SQLite.

4.1 Database Modeling

4.1.1 Entity Summary

Entity	Table	Purpose
User	`users`	Account credentials and identity
Task	`tasks`	Primary task items owned by a user
Subtask	`subtasks`	Nested items belonging to one task

4.1.2 Entity Relationship Diagram

Figure 12 shows a compact **Crow's Foot** (IE) style ERD. Each `task` references exactly one `user` (`user_id`); each `user` owns zero or more `tasks`. Each `subtask` references exactly one `task` (`task_id`); each `task` has zero or more `subtasks`.

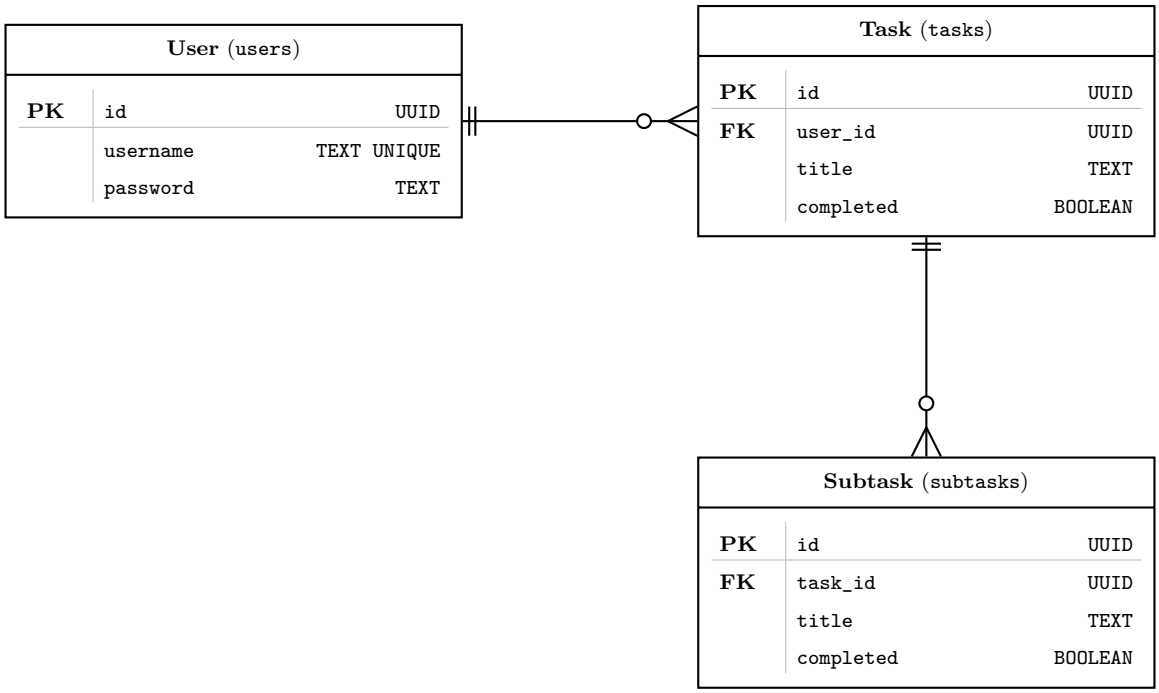


Figure 12: Crow's Foot ERD

4.1.3 Data Dictionary

The ERD above is the canonical model. Table 1 maps each logical entity to its SQLite columns, types, and constraints.

Table 1: SQLite data dictionary

Table	Column	Type	Constraints / Notes
users	id	UUID	PRIMARY KEY, auto-generated (GenerationType.UUID)
users	username	TEXT	NOT NULL, UNIQUE
users	password	TEXT	NOT NULL
tasks	id	UUID	PRIMARY KEY, auto-generated (GenerationType.UUID)
tasks	user_id	UUID	NOT NULL; REFERENCES users(id)
tasks	title	TEXT	NOT NULL
tasks	completed	BOOLEAN	NOT NULL, DEFAULT false
subtasks	id	UUID	PRIMARY KEY, auto-generated (GenerationType.UUID)
subtasks	task_id	UUID	NOT NULL; REFERENCES tasks(id)
subtasks	title	TEXT	NOT NULL
subtasks	completed	BOOLEAN	NOT NULL, DEFAULT false

4.1.4 Integrity Rules

- `users.username` must be unique.
- Every `task` must reference a valid `user_id`.
- Every `subtask` must reference a valid `task_id`.
- Deleting a task cascade-deletes its subtasks (enforced in service layer).

Team Work Records

Todo Management Application

Revature — Full-Stack SDLC Project

The DJs

Prepared by

Jaehoon Song
Daniel Patience
Jacqueline Lainhart

Document ID: TMA-TWR-P1
Version: 0.3
Status: Draft
Issue date: June 26, 2026

5 Team Work Records

This portion records how the team measures completion and tracks ongoing work. The Definition of Done establishes criteria for finished deliverables; the task backlog summarizes session outcomes, backlog status, and the GitHub Issues workflow used to assign remaining items.

5.1 Definition of Done

The team established the following criteria on **10 June 2026**. A Phase 1 checkpoint or deliverable is considered *done* when all applicable items below are satisfied.

5.1.1 Documentation Criteria

- Content reflects decisions made in team discussion (see `discussions/` in the repository).
- Technical writing is merged into `docs/module/` (modular `.tex` files) and compiles without errors.
- Section owners verify accuracy before the item is marked complete in the checkpoint list.

5.1.2 Collaboration Criteria

- Work is committed on a feature branch and merged via pull request—not pushed directly to `main`.
- Discussion notes and LaTeX documentation may land on separate branches and are integrated independently.
- At least one teammate reviews the pull request before merge.

5.1.3 Checkpoint Exit Criteria

- **Task Breakdown** — All four epics documented with task IDs (Section 2.1).
- **Database Modeling** — ERD, data dictionary, and integrity rules agreed upon (Section 4.1).
- **API Contract** — Every endpoint from the task breakdown has request/response bodies and HTTP status codes (Section 2.2).
- **Task Backlog** — Remaining Phase 1 items tracked in GitHub Issues or Projects (Section 5.2).

5.1.4 Phase 1 Completion

As of 11 June 2026, all Phase 1 checkpoints are complete. The team may proceed to Phase 2 once all members have reviewed Sections 2.1–2.2.

5.1.5 Phase 2 Progress

Phase 2 completed 12 June 2026. All deliverables meet the documentation and collaboration criteria and are recorded in the task backlog.

- **UI Wireframing** — Complete: Login, Register, Dashboard, and navigation flow (Section 3.2).
- **Component Design** — Complete: routes, component tree, and service layer agreed (Section 5.2).
- **LaTeX refactor** — Complete: modular `docs/module/` layout, section introductions, expanded sequence diagrams.
- **System Architecture** — Complete: data-flow documentation merged.
- **Security Planning** — Complete: JJWT and auth-interceptor design agreed.

5.1.6 Phase 3 Completion

Phase 3 (Backend Development) completed 22 June 2026. All checkpoints satisfied:

- **Project Scaffolding** — Spring Boot 4.1.0, Gradle Kotlin DSL, SQLite configured. #46, #45, #44
- **Data Layer** — JPA entities (User, Task, Subtask), repositories with UUID keys. PR #49
- **Security Implementation** — JWT via JJWT 0.13.0, HandlerInterceptor, PasswordValidator. #51, PR #40
- **Business Logic** — Registration, login, task CRUD, subtask CRUD with ownership enforcement. #11, #50, PR #55, PR #56
- **Design Validation** — All 11 endpoints verified via cURL (45 test cases, all passing).

5.1.7 Phase 4 Completion

Phase 4 (Frontend Development) completed 25 June 2026. All checkpoints satisfied:

- **Project Scaffolding** — Angular 22 standalone components, Vitest, proxy config. #54
- **Service Layer** — TaskService, SubtaskService, AuthService with signal-based state. PR #62, PR #63
- **Auth Integration** — Functional interceptor, route guard, login/register components. PR #59, PR #60, PR #58
- **Task Management UI** — Dashboard with inline edit, expand/collapse subtasks, CRUD. PR #64
- **Design Validation** — UI matches wireframes; end-to-end workflow verified. PR #67

5.1.8 Phase 5 Status

Phase 5 (Presentation) is *ready* as of 26 June 2026.

- **Functional Demo** — All user stories operational; happy-path scenarios verified.
- **Technical Review** — Data flow, schema design, and architecture documented.
- **Build Verification** — Clean build confirmed (`gradlew build`, `ng build`).
- **Post-Mortem** — Agile process, Git workflow, and blockers documented in project records.

5.2 Task Backlog

Use GitHub Issues and Github Project to track actionable items. Phase 1 completed **11 June 2026**; Phase 2 completed **12 June 2026**; Phase 3 completed **22 June 2026**; Phase 4 completed **25 June 2026**; Phase 5 ready **26 June 2026**. Session summaries below record how each phase was closed out.

5.2.1 Session Summary — 10 June 2026

Activity	Outcome
LaTeX documentation initiated	Jaehoon Song set up the `docs/` structure and `main.tex` driver.
Team meeting — task breakdown	All members decomposed user stories into epics and task IDs; documented in Section 2.1.
Initial ERD	Jaehoon Song drafted the Crow's Foot ERD and SQLite data dictionary (Section 4.1).
Repository integration	Each member used feature branches to merge discussion notes and LaTeX updates separately into `main`.
Definition of Done	Team agreed on completion criteria (Section 5.1).
Task Backlog	Initial Phase 1 progress and GitHub workflow recorded (Section 5.2).

5.2.2 Session Summary — 11 June 2026

Activity	Outcome
API Contract	Daniel Patience and Jacqueline Lainhart drafted payloads and status codes; documented in Section 2.2.
HTTP request/response documentation	Jaehoon Song formatted API exchanges as HTTP traces (`httpexchange` layout); Section 2.2.
Phase 1 completion	All Phase 1 checkpoints satisfied; team cleared to begin Phase 2 (Section 5.1).

5.2.3 Session Summary — 12 June 2026

Activity	Outcome
LaTeX documentation refactor	Jaehoon Song reorganized content into `docs/module/`, added section introductions in `main.tex`, and expanded Epic 3/4 sequence diagrams (Section 3.1).
UI Wireframing	Login, Register, Dashboard, and screen-navigation wireframes added to `docs/Wireframes/` and documented in Section 3.2.
Component Design	Team agreed Angular routes (`/`, `/register`, `/dashboard`), component hierarchy (`AppComponent`, auth views, `TodoItemComponent` with nested subtasks), and services (`AuthService`, `TodoService`, `SubtaskService`, `AuthInterceptor`, `AuthGuard`).
Phase 2 completion	All Phase 2 checkpoints and deliverables satisfied; team cleared to begin Phase 3 (Section 5.1).

5.2.4 GitHub Workflow

1. Create a branch from `main` for the work item (discussion notes or documentation).
2. Commit changes locally; open a pull request when ready for review.
3. A teammate reviews and approves; merge into `main`.
4. Open or update a GitHub Issue for any remaining task IDs; link the issue to the pull request where applicable.

5.2.5 Backlog Status — Phase 1 (complete 11 June 2026)

Item	Status	Notes
Task Breakdown (Section 2.1)	Complete	Documented from team meeting discussion; 10 June 2026.
Database Modeling (Section 4.1)	Complete	Initial ERD and data dictionary; 10 June 2026.
API Contract (Section 2.2)	Complete	Payloads by Daniel Patience and Jacqueline Lainhart; HTTP req/res traces by Jaehoon Song; 11 June 2026.
Definition of Done (Section 5.1)	Complete	Criteria established; 10 June 2026.
GitHub Issues / Project board	Complete	Workflow recorded; issues to be opened as Phase 3 work is assigned.

5.2.6 Backlog Status — Phase 2 (complete 12 June 2026)

Item	Status	Notes
System Architecture (Appendix **??**)	Complete	Data flow documented in `discussions/phase-2-deliverables.md`; 12 June 2026.
Security Planning (Appendix **??**)	Complete	JJWT and auth approach agreed; 12 June 2026.
UI Wireframing (Section 3.2)	Complete	Four wireframes under `docs/Wireframes/`; 12 June 2026.
Component Design (Appendix **??**)	Complete	Routes, component tree, and services agreed; 12 June 2026.
LaTeX documentation refactor	Complete	Modular `docs/module/` structure and section intros; 12 June 2026.
API Specification (Section 2.2)	Complete	Finalized for implementation; 12 June 2026.
Database Schema (Section 4.1)	Complete	ERD and data dictionary signed off; 12 June 2026.

5.2.7 Session Summary — 16 June 2026

Activity	Outcome
Spring Boot scaffolding	Kiro-assisted project initialization with Gradle Kotlin DSL, SQLite, JPA. #46
Ignore rules and steerings	Configured `.gitignore` and Kiro steering files for team workflow. #45, #44
Password requirements	Daniel Patience documented password validation rules. PR #40

5.2.8 Session Summary — 19 June 2026

Activity	Outcome
JPA entities and repositories	Jacqueline Lainhart ran Kiro Phase 3 tasks for entity setup. PR #49
Feature issues created	Opened spec-driven issues for registration, auth, and task CRUD. #50, #51, #52

5.2.9 Session Summary — 22 June 2026

Activity	Outcome
Registration spec and implementation	Jacqueline Lainhart delivered registration spec and ran tasks. PR #55, #11 closed
Task/Subtask CRUD	Daniel Patience implemented TaskService, SubtaskService, controllers. PR #53, PR #56
Frontend issue opened	Angular frontend work tracked under single issue. #54

5.2.10 Session Summary — 23 June 2026

Activity	Outcome
Registration tasks completed	Jacqueline Lainhart fixed DTO/bcrypt issues and ran registration tasks. PR #57, PR #58
Authentication spec and draft	Jaehoon Song created Kiro spec for US02, drafted auth feature. PR #59, PR #60
Entity alignment	Daniel Patience renamed userId fields, aligned folder structure. PR #61
Frontend specs	Jaehoon Song added client01 auth-integration spec; Daniel added dashboard specs. PR #62, PR #63

5.2.11 Session Summary — 24–25 June 2026

Activity	Outcome
Full frontend implementation	Jacqueline Lainhart delivered Angular app: auth, dashboard, services. PR #64
Build fixes	Jaehoon Song fixed ignore rules to exclude DB binaries. PR #66
UI/UX polish	Jacqueline Lainhart finalized styling and responsiveness. PR #67
Phase 4 complete	End-to-end workflow verified; all user stories functional.

5.2.12 Backlog Status — Phase 3 (complete 22 June 2026)

Item	Status	Notes
Project Scaffolding	Complete	Spring Boot 4.1.0, Gradle, SQLite; #46
Data Layer	Complete	User, Task, Subtask JPA entities; PR #49
Security	Complete	JWT + HandlerInterceptor; PR #60
Business Logic	Complete	All CRUD services; PR #56
Design Validation	Complete	45 curl tests passing; API contract synced 26 June 2026

5.2.13 Backlog Status — Phase 4 (complete 25 June 2026)

Item	Status	Notes
Angular Scaffolding	Complete	Angular 22, standalone components; #54
Service Layer	Complete	TaskService, SubtaskService; PR #63
Auth Integration	Complete	AuthService, interceptor, guard; PR #62
Task Management UI	Complete	Dashboard with inline edit, subtask expand; PR #64
Design Validation	Complete	Matches wireframes; PR #67

5.2.14 Backlog Status — Phase 5 (ready 26 June 2026)

Item	Status	Notes
Functional Demo	Ready	All user stories operational
Technical Review	Ready	Data flow, schema, architecture documented
Build Verification	Ready	`gradlew build` and `ng build` confirmed
Post-Mortem	Ready	Agile process and blockers recorded in session summaries above